ASSIGNMENT 2 – CO2

C++ Programming: Functions and Object-Oriented Concepts

Name: Sahhanaa Shree Thiyagarajan
Register Number: 192511283
Course: C++ Programming
Assignment: Assignment 2 – CO2
Submission Date: 31 October 2026
GitHub Repository: https://github.com/sahhanaa44/DSA0101-192511283A-Assignment2

INTRODUCTION

The objective of this assignment is to evaluate the implementation and effectiveness of functions and object-oriented programming concepts in C++. The assignment focuses on call by value, call by reference, pointers, return by reference, default arguments, function overloading, classes, access specifiers, arrays within classes, static members, arrays of objects, friend functions, and inline functions.

Each problem is designed around a real-world scenario. The solutions are evaluated based on correctness, memory utilization, computational complexity, maintainability, and adaptability to changing requirements.

Q1 – CALL BY VALUE, CALL BY REFERENCE AND CALL BY POINTER

Problem Context: Hostel Fee Concession Updater

Aim

To implement three parameter-passing mechanisms in C++ and evaluate their effects on the original variable.

Problem Description

A hostel management system provides fee concessions based on student eligibility:

- Merit: 10%
- Sports: 15%
- Single Parent: 25%

The system must correctly update the student's hostel fee after applying the appropriate concession.

Implementation

Three functions are implemented:

Call by Value: A copy of the original fee is passed to the function. Any modification affects only the local copy.

Call by Reference: The function receives a reference to the original fee. Modifications directly affect the caller's variable.

Call by Pointer: The function receives the address of the fee and modifies its value through dereferencing.

For a fee of ₹45,000 with a 10% concession:

Concession = 45000 × 10 / 100 = ₹4,500

Updated fee = 45000 − 4500 = ₹40,500

Predict the Output

For the provided program involving change(), bigger() and show():

```
None
1 12 13
1 100
16 13 6
```

Explanation:

The change() function receives a by value, b by reference and c by pointer. Therefore, only b and c change in the caller.

The bigger() function returns a reference to the larger variable. The assignment of 100 modifies c.

The show() function uses default arguments:

- show(1) → 1 + 5 + 10 = 16
- show(1,2) → 1 + 2 + 10 = 13
- show(1,2,3) → 1 + 2 + 3 = 6

Complete the Code

The missing statement in applyConcession() is:

```
None
fee -= fee * percent / 100;
```

Expected Output:

```
None
after call by value   : 45000
after call by reference: 40500
```

Inquiry and Design Decision

In call by value, the parameter is a separate local object. In call by reference, the parameter is another name for the caller's object. In call by pointer, a pointer parameter holds an address through which the caller's object can be modified.

A const reference is preferable when a function needs to read a large object without copying it and must not modify it.

For multiple concessions, returning the updated fee is recommended because it makes state changes explicit and allows validation before assignment.

Alternative Concept and Complexity Analysis

Method	Time Complexity	Auxiliary Space	Modifies Original
Call by Value (long long)	O(1)	O(1)	No
Call by Reference	O(1)	O(1)	Yes
Call by Pointer	O(1)	O(1)	Yes
Return Updated Fee	O(1)	O(1)	Only after assignment

For a large Student object of size S, pass by value requires O(S) copying time and additional space, whereas passing by reference or const reference avoids copying the object.

Result

The program demonstrates that reference and pointer parameters can directly modify caller-owned data, while call by value cannot.

Execution Evidence: [Insert Q1 screenshots]

Q2 – RETURN BY REFERENCE AND ITS DANGERS

Problem Context: Examination Marks Register

Aim

To demonstrate returning references from functions, modifying array elements through references, and identifying the dangers of dangling references.

Implementation

An array stores examination marks. A function identifies the highest mark and returns a reference to the corresponding element.

```
None
int& highest(int marks[], int n) {
    int index = 0;

    for (int i = 1; i < n; i++) {
        if (marks[i] > marks[index])
            index = i;
    }

    return marks[index];
}
```

The returned reference can appear on the left-hand side of an assignment:

```
None
highest(marks, n) += 2;
```

This directly increases the highest mark stored in the array.

Dangling Reference

A function must not return a reference to a local automatic variable:

```
None
int& invalidReference() {
    int value = 50;
    return value;
}
```

The variable's lifetime ends when the function returns. Accessing the returned reference causes undefined behavior.

The compiler may display a warning such as:

```
None
warning: reference to local variable 'value' returned
```

Note: The exact warning depends on the compiler and version.

Reference vs Const Reference vs Copy

Return Type	Modifiable Through Result	Main Property
int&	Yes	Refers to existing object
const int&	No	Read-only access to existing object
int	No effect on original	Independent returned value

Changed Condition: Growing Vector

When a std::vector grows beyond its current capacity, reallocation may move its elements to new memory locations.

References and pointers to the old elements can become invalid.

Returning an index can be safer because it does not directly store an element's memory address. However, indices can also become invalid or refer to different elements after insertion, removal or reordering.

Alternative Concept and Complexity Analysis

Approach	Search Time	Extra Space	Safety
Reference	O(n)	O(1)	May dangle
Pointer	O(n)	O(1)	May become invalid
Index	O(n)	O(1)	Avoids pointer invalidation but requires validation

Design Decision

For a growing vector, returning an index and accessing the vector only after checking that the index is valid is preferable.

Result

Returning a reference enables direct modification of existing data, but its safety depends on the lifetime and stability of the referenced object.

Execution Evidence: [Insert Q2 output and compiler warning screenshots]

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING

Problem Context: Courier Charge Calculator

Aim

To evaluate default arguments and function overloading by developing a courier charge calculator without ambiguous function calls.

Problem Description

Courier charges depend on parcel weight, express delivery and insurance.

The original design contains overlapping functions:

```
None
charge(double weight);
charge(double weight, bool express);
charge(double weight, bool express = false,
    bool insured = false);
```

A call such as charge(2.0) can match all three declarations because the missing arguments can be supplied through defaults.

This produces an ambiguous function call error.

Redesigned Solution

The revised design uses different parameter types to distinguish the overloads:

```
None
charge(int grams);
charge(double kilograms, bool express = false,
    bool insured = false);
charge(const Parcel& parcel);
```

This supports short calls and avoids overlapping signatures.

Predict the Overload Selection

Function Call	Selected Overload
charge(2)	Integer/grams
charge(2.0)	Double/kilograms
charge(2.0f)	Double/kilograms through conversion

charge(2.5, true)	Double with express
charge(2.5, true, true)	Double with express and insurance
charge(parcel)	Parcel object

The float argument is converted to double because there is no exact float overload.

Inquiry

C++ overload resolution generally prefers:

1. Exact match
2. Promotion
3. Standard conversion

These rules are applied to the complete candidate function signature; some calls can remain ambiguous when candidates have competing conversion sequences.

Default arguments must be supplied from the right because omitted arguments must form a trailing sequence.

Alternative Concept

A single Parcel structure can store the weight and optional delivery information.

This makes it easier to introduce additional options such as fragile handling, priority shipping or international delivery.

Complexity Analysis

Overload selection occurs during compilation and introduces no additional runtime dispatch cost.

Design	Runtime Complexity	Parameter Copying
Overloaded Functions	O(1) for simple charge calculation	Small scalar parameters
Parcel by Value	O(1) for fixed-size structure	Copies entire structure
Parcel by Const Reference	O(1)	Avoids object copy
Function Template	Depends on implementation	Depends on parameter type

Design Decision

Overloading is suitable when inputs represent genuinely different forms of data. A Parcel structure is preferable when the number of optional features increases significantly.

Result

The redesigned overloads provide readable calls while eliminating ambiguity.

Execution Evidence: [Insert Q3 screenshots]

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT

Problem Context: Savings Account Mini-Statement

Aim

To design a SavingsAccount class using private data members, public member functions and an array to maintain recent transactions.

Class Design

The class contains:

Private Members:

- Account number
- Account balance
- Array of 10 transactions
- Transaction count

Public Members:

- deposit()
- withdraw()
- miniStatement()

Some member functions are defined inside the class, while others use the scope resolution operator :: outside the class.

Working Principle

Whenever a customer deposits or withdraws money, the transaction is recorded.

Only the latest ten transactions are maintained.

When the eleventh transaction arrives, the oldest transaction is removed from the stored history.

Inquiry

The transaction array must remain private to prevent unauthorized modification and maintain data integrity.

Two methods can maintain the latest transactions.

Array Shifting: When full, each element is shifted left and the newest transaction is inserted at the end.

Circular Buffer: A starting index is maintained and the oldest element is overwritten when capacity is reached.

Complexity Comparison

Operation	Shifting Array	Circular Buffer	Deque
Add transaction when full	O(k)	O(1)	O(1)
Retrieve all transactions	O(k)	O(k)	O(k)
Storage	O(k)	O(k)	O(k)

Here, k represents the transaction history capacity.

Changed Requirement

The bank increases the history limit from 10 to 50 transactions.

For a fixed-size array, the capacity must be updated. A circular buffer still requires the same basic operations, whereas a shifting implementation may perform more work as the capacity increases.

A std::deque allows the limit to change without redesigning the storage algorithm.

Design Decision

A circular buffer is recommended when the history capacity is fixed because insertion remains O(1), even when the buffer is full.

For frequently changing storage requirements, a deque provides greater flexibility.

Result

The class demonstrates encapsulation and transaction management while comparing alternative data-storage strategies.

Execution Evidence: [Insert Q4 screenshots]

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS

Problem Context: College Bus Seat Tickets

Aim

To implement static data members, static member functions and arrays of objects for managing college bus seat allocation.

Problem Description

A college bus has a limited number of seats.

Each ticket receives the next available seat number. Once the capacity is reached, subsequent passengers are placed on the waiting list.

The total number of issued seats must be accessible without referencing any particular ticket object.

Class Design

The BusTicket class contains:

- Static integer issued

- Static constant CAPACITY
- Individual seat number
- Passenger name
- Static member function count()

The static variable is defined outside the class:

```
None
int BusTicket::issued = 0;
```

Only one copy of this variable exists for the entire class.

Predict the Output

For the given Token program:

```
None
101 103 104 4
```

Explanation:

The static counter begins at 100.

The array creates three objects, assigning IDs 101, 102 and 103.

The additional object receives ID 104.

The constructor executes four times, and issued() returns 104 − 100 = 4.

Complete the Code

TODO 1:

```
None
if (issued < CAPACITY) {
    seat = ++issued;
}
```

TODO 2:

```
None
return issued;
```

Expected Output:

```
None
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Static Member Behaviour

A static member function can access static members directly.

It cannot use this because it is not called on a particular object.

Non-static members belong to individual objects and therefore require an object reference.

When tickets are created inside a function called twice, the static counter persists between function calls.

Once the capacity has been reached, newly created tickets remain on the waiting list unless the allocation state is explicitly reset.

Alternative Concept and Complexity Analysis

Approach	Allocation Time	Counter Storage	Multiple Bus Support
Static Counter	O(1)	O(1)	Requires redesign for independent buses
Global Counter	O(1)	O(1)	Poor encapsulation
TicketOffice Object	O(1)	O(1) per office	Supports independent buses

Design Decision

A static counter is suitable when all tickets belong to one shared allocation system.

For multiple buses, separate TicketOffice objects are preferable because each bus can maintain independent capacity and allocation information.

Result

The implementation demonstrates shared class-level data, automatic seat allocation and waiting-list handling.

Execution Evidence: [Insert Q5 screenshots]

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS

Problem Context: Train Journey Planner

Aim

To implement friend functions for comparing private time values and evaluate the performance impact of inline functions.

Problem Description

A train journey planner stores hours and minutes in a Time class.

The application must calculate the difference between two times and determine which time occurs earlier.

The comparison functions need access to the private hour and minute fields of two objects.

Implementation

The class declares two friend functions:

```
None
friend int minutesBetween(const Time& a,
                          const Time& b);

friend bool earlier(const Time& a,
```

```
        const Time& b);
```

A friend function is not a member function, but it is permitted to access the class's private and protected members.

Complete the Code

TODO 1:

```
None
int difference =
    (a.h * 60 + a.m) - (b.h * 60 + b.m);

return difference < 0 ? -difference : difference;
```

TODO 2:

```
None
return (a.h * 60 + a.m) <
       (b.h * 60 + b.m);
```

Expected Output

```
None
205 false
```

The time difference between 9:45 AM and 1:10 PM is:

13 × 60 + 10 = 790 minutes

9 × 60 + 45 = 585 minutes

Difference = 790 − 585 = 205 minutes.

Since 1:10 PM is not earlier than 9:45 AM, the comparison returns false.

Correctness Tests

Time A	Time B	Difference	Earlier(A,B)
09:45	13:10	205	true
12:00	12:00	0	false
11:59	12:01	2	true
15:30	08:00	450	false

Inline Function Analysis

The inline keyword permits a function definition to appear in multiple translation units under the appropriate rules and serves as a hint that call overhead might be avoided.

However, the compiler is not required to inline the function.

Modern optimizing compilers can inline functions even without the keyword.

Performance Measurement

The assignment requires 10,000,000 function calls using two compiler optimization levels.

Compilation Commands:

```
None
g++ -std=c++17 -O0 benchmark.cpp -o benchmark_O0
g++ -std=c++17 -O2 benchmark.cpp -o benchmark_O2
```

Measurement Table — Fill After Execution

Configuration	Time for 10 Million Calls
Inline, -O0	[Measured time]
Non-inline, -O0	[Measured time]
Inline, -O2	[Measured time]
Non-inline, -O2	[Measured time]

At -O0, function-call overhead may be more visible. At -O2, the compiler may optimize both versions similarly.

Actual measurements depend on compiler version, processor, optimization decisions and benchmark design.

Alternative Design

The same functionality can be implemented using:

- Member functions such as a.minutesTo(b)
- Public getter functions
- Friend functions

All three approaches have O(1) time complexity.

Design Decision

Friend functions are appropriate when the operation treats both objects equally.

However, getters or member functions may be preferable when reducing privileged access to internal data is a design priority.

Making the private time fields public is not recommended because it allows unrestricted changes that may violate valid time constraints.

Result

The implementation demonstrates friend-function access and constant-time comparison. The benchmark is intended to evaluate the effects of compiler optimization and function inlining.

Execution Evidence: [Insert Q6 output and benchmark screenshots]

Q7 – DIAGNOSE, REDESIGN AND VALIDATE

Selected Program: Q4 – Savings Account Mini-Statement

A. Diagnose the Limitation

The original savings account implementation maintains the latest transactions using a fixed array and shifts elements whenever the history becomes full.

For a capacity of k transactions, shifting requires O(k) time for each insertion after the array reaches capacity.

As the number of retained transactions increases, this operation becomes more expensive.

Evidence: [Insert original implementation output and measured insertion time]

B. Define the Design Objective

The objective is to redesign transaction storage so that inserting a new transaction takes O(1) time, regardless of the number of transactions already stored.

The revised implementation must:

- Retain only the most recent k transactions.
- Preserve the correct chronological order when displaying history.
- Avoid shifting the entire array during insertion.
- Maintain O(k) storage complexity.

C. Design the Revised Solution

A circular buffer is introduced.

The revised class maintains:

- An array of transactions
- A starting index
- A transaction count
- A fixed capacity

When the array becomes full, the oldest transaction is overwritten and the starting index advances.

Pseudocode:

```
None
ADD_TRANSACTION(value):

    IF count < capacity:
        position = (start + count) % capacity
        transactions[position] = value
```

```
        count = count + 1

    ELSE:
        transactions[start] = value
        start = (start + 1) % capacity
```

Class Design:

```
None
SavingsAccount
--------------------------------
- accountNumber
- balance
- transactions[CAPACITY]
- start
- count
--------------------------------
+ deposit(amount)
+ withdraw(amount)
+ addTransaction(amount)
+ miniStatement()
```

The revised solution requires two additional integer fields but avoids repeated array shifting.

D. Predict Before Implementing

The circular buffer is expected to reduce insertion complexity from O(k) to O(1).

For larger transaction-history capacities, the redesigned implementation is expected to perform better when repeatedly inserting into a full buffer.

For small capacities, the difference may be negligible because shifting a small number of elements is inexpensive.

E. Implement and Validate

Both implementations should be tested using the same transaction sequences and capacity settings.

Functional Validation:

For a history capacity of five, insert transactions:

```
None
100, 200, 300, 400, 500, 600, 700
```

Both implementations should retain:

```
None
300, 400, 500, 600, 700
```

Performance Comparison — Fill After Execution

Test	Original Shifting Array	Circular Buffer
Correct last k transactions	[Pass/Fail]	[Pass/Fail]
Insertions with k = 10	[Time]	[Time]
Insertions with k = 50	[Time]	[Time]
Insertions with k = 1000	[Time]	[Time]
Insertion Complexity	O(k) when full	O(1)
Storage Complexity	O(k)	O(k)

The timings must be obtained by running both versions under comparable conditions.

F. Defend the Final Design

The circular-buffer design improves the asymptotic cost of inserting transactions into a full history.

It removes the need to shift existing entries, making insertion time independent of the history capacity.

The trade-off is additional index-management logic. Displaying the transactions in chronological order requires modular indexing.

The original shifting-array implementation may remain preferable for very small capacities where simplicity is more important than insertion performance.

Final Evaluation: The redesign meets the theoretical O(1) insertion objective. Whether it achieves a measurable performance improvement on the test system must be determined from the recorded benchmark results.

Execution Evidence: [Insert original-versus-redesigned screenshots and timing results]

OVERALL COMPLEXITY SUMMARY

Question	Main Concept	Time Complexity	Auxiliary Space
Q1	Parameter Passing	O(1) for scalar fee	O(1)
Q2	Finding Highest Mark	O(n)	O(1)
Q3	Courier Charge Calculation	O(1) for fixed-size inputs	O(1)
Q4	Transaction Storage	O(k) shifting, O(1) circular insertion	O(k)
Q5	Static Seat Allocation	O(1) per ticket	O(1) per ticket
Q6	Time Comparison	O(1)	O(1)
Q7	Circular Buffer Redesign	O(1) insertion	O(k)

CONCLUSION

This assignment explored the practical applications of functions and object-oriented programming in C++ through seven real-world problems.

The implementations demonstrate the importance of choosing appropriate parameter-passing mechanisms, maintaining valid references, avoiding ambiguous function overloads, enforcing encapsulation, using static members correctly, and evaluating friend and inline functions.

The redesign exercise highlights how an implementation that works correctly for small inputs may become inefficient when requirements change.

Overall, the assignment emphasizes that good software design requires not only correct output but also safe memory access, efficient algorithms, maintainable interfaces and evidence-based evaluation.

EXECUTION AND SUBMISSION DETAILS

Programming Language: C++17
Compiler: [Enter actual compiler and version]
Development Environment: Visual Studio Code
Version Control: Git and GitHub
Operating System: Windows

GitHub Repository:
https://github.com/sahhanaa44/DSA0101-192511283A-Assignment2